

Scallion Pancake Party

Bohan กำลังจะจัดปาร์ตี้ เนื่องจากเขาชอบแพนเค้กต้นหอมมาก เขาตัดสินใจที่จะซื้อจากร้านใกล้ๆ เพื่อปาร์ตี้นี้ ร้านขายแพนเค้กต้นหอมแตกต่างกันทั้งหมด N สส เรียงตั้งแต่ 0 ถึง $N - 1$ Bohan ตัดสินใจที่จะซื้อแพนเค้กต้นหอมจำนวน N แผ่น สำหรับแต่ละสส รวมเป็นแพนเค้กต้นหอมจำนวน N^2 แผ่น แพนเค้กแต่ละแผ่นจะถูกแบ่งเป็น K ชิ้นเท่า ๆ กัน ก่อนจะถูกนำรวมใส่ถุงที่เหมือนกัน แผ่นละหนึ่งถุง ถุงเหล่านี้จะถูกนับเป็นหมายเลขตั้งแต่ 0 ถึง $N^2 - 1$ ให้ $F[i]$ แทนรสของแพนเค้กต้นหอมสำหรับถุง i

ระหว่างปาร์ตี้ Bohan เชิญแขกผู้ร่วมงานให้เล่นเกม ที่มีขั้นตอนดังนี้

ในขั้นแรก Bohan จะเลือกแขกจำนวน N คน และพาไปยังห้องที่มีหมายเลข $0, 1, \dots, N - 1$ โดยที่แต่ละห้องจะมีแขกหนึ่งคนพอดี แขกที่เหลือจะรออยู่ด้านนอกจนกระทั่งจบเกม ห้องทั้งหมดดูเหมือนกัน ดังนั้น แขกจะไม่สามารถระบุได้ว่าพวกเขาอยู่ห้องใดกันแน่

ในขั้นถัดไป สำหรับแต่ละห้อง i ($0 \leq i < N$), Bohan จะเข้าไปในห้อง i และจะบอกกับแขกที่อยู่ห้องนั้นแบบลับ ๆ ถึงจำนวนเต็ม $P[i]$ ที่แทน **รสต้องห้าม** เขาอาจจะบอกหรือไม่บอกกับแขกก็ได้ว่าแขกอยู่ห้องใด Bohan รับประกันว่า $[P[0], P[1], \dots, P[N - 1]]$ จะเป็นการเรียงสับเปลี่ยน (permutation) ของ $[0, 1, \dots, N - 1]$

จากนั้น จะมีการเล่นเกมจำนวน N รอบ ในรอบที่ i Bohan จะนำถุง $0, 1, \dots, N^2 - 1$ เข้าไปยังห้อง $i - 1$ ทีละถุง ตามลำดับ เมื่อแขกที่ห้อง $i - 1$ ได้รับถุง j เขาจะได้ทราบ:

- $F[j]$ รสของแพนเค้กต้นหอมในถุง j และ
- จำนวนของชิ้นแพนเค้กที่เหลือในถุง

ก่อนที่จะได้รับถุงถัดไป แขกที่ห้อง $i - 1$ จะต้องตัดสินใจว่าจะกินแพนเค้กที่ชิ้นจากถุงปัจจุบันที่ได้รับ จำนวนชิ้นที่สามารถกินได้ขึ้นกับสถานการณ์:

- ถ้า $P[i - 1] \neq F[j]$, แขกสามารถนำชิ้นแพนเค้กที่ชิ้นก็ได้ออกจากถุงและกินมันเข้าไป
- ถ้า $P[i - 1] = F[j]$, แขกจะไม่สามารถอนุญาตให้กินแพนเค้กได้เลย

โปรดทราบไว้ว่า แขกจะไม่ทราบลำดับ $F[0], F[1], \dots, F[N^2 - 1]$ ล่วงหน้า

หลังจากการเล่น N รอบ แขกผู้ร่วมงานด้านนอกจะได้รับลำดับของถุง และจะทราบรสของแพนเค้กต้นหอมในถุงนั้น และจำนวนชิ้นแพนเค้กที่เหลือในถุง ด้วยข้อมูลดังกล่าว พวกเขาจะต้องระบุ $P[0], P[1], \dots, P[N - 1]$ ให้ได้

แขกผู้ร่วมงานทั้งหมดสามารถสื่อสารกันได้ก่อนเกมจะเริ่ม พวกเขายังทราบค่าของ N และ K ล่วงหน้า เป้าหมายของคุณก็คือเขียนกลยุทธ์ที่ทำให้แขกผู้ร่วมงานด้านนอกสามารถระบุค่า $P[0], P[1], \dots, P[N - 1]$ ได้อย่างถูกต้องเสมอ

Implementation details

คุณจะต้องเขียนฟังก์ชันจำนวนสามฟังก์ชัน.

คุณจะต้องเขียนฟังก์ชันทั้งสองต่อไปนี้ สำหรับแขกที่ห้อง $0, 1, \dots, N - 1$

```
void init(int N, int K, int p, int r)
```

สมมติว่าแขกอยู่ที่ห้อง i .

- N : จำนวนรสของแพนเค้กต้นหอม.
- K : จำนวนชั้นของแพนเค้กในแต่ละถาดก่อนที่เกมจะเริ่ม.
- $p = P[i]$: รสที่ห้ามกินของห้อง i .
- ถ้า Bohan ตัดสินใจว่าจะบอกแขกว่าอยู่ที่ห้องไหน, ค่าของ $r = i$. ถ้าไม่เช่นนั้น, $r = -1$.

```
int strategy(int b, int f, int s)
```

ถ้าแขกอยู่ที่ห้อง i .

- b : หมายเลขถาด.
- $f = F[b]$: รสของแพนเค้กต้นหอมในถาด b .
- s : จำนวนชั้นแพนเค้กที่เหลือในถาด b .
- ฟังก์ชันจะต้องคืนจำนวนเต็มไม่เป็นลบ x แทนจำนวนชั้นแพนเค้กที่แขกตัดสินใจจะกิน.
 - ถ้า $f \neq P[i]$, จะต้องรับประกันว่า $0 \leq x \leq s$.
 - ถ้า $f = P[i]$, แล้วจะต้องรับประกันว่า $x = 0$.

คุณจะต้องเขียนฟังก์ชันต่อไปนี้สำหรับแขกผู้ร่วมงานด้านนอกห้อง.

```
std::vector<int> guess(int N, int K, std::vector<int> F,  
                      std::vector<int> S)
```

- N : จำนวนรสของแพนเค้กต้นหอม.
- K : จำนวนชั้นของแพนเค้กต้นหอมในแต่ละถาด ก่อนที่จะเริ่มเกม.
- F : อาร์เรย์ขนาด N^2 โดยที่ $F[i]$ ระบุรสของแพนเค้กต้นหอมในถาด i .
- S : อาร์เรย์ขนาด N^2 โดยที่ $S[i]$ ระบุจำนวนชั้นของแพนเค้กที่เหลือในถาด i หลังการเล่นเกมผ่านไปแล้ว N รอบ.
- ฟังก์ชันจะต้องคืนอาร์เรย์ P ที่มีความยาว N โดยที่ $P[i]$ ระบุจำนวนเต็มที่ให้กับแขกในห้อง i .

แต่ละชุดข้อมูลทดสอบ จะประกอบไปด้วยเกมจำนวน T เกม ที่เล่นแบบเป็นอิสระต่อกัน

ระหว่างการตรวจ, สำหรับแต่ละเกมใน T เกม จะมีโปรเซสทำงานจำนวน $N + 1$ โปรเซส สำหรับแต่ละ i ที่ $0 \leq i < N$, โปรเซส i จะแทนแขกผู้ร่วมงานในห้อง i โปรเซส N จะแทนแขกผู้ร่วมงานด้านนอก สำหรับแต่ละ i ที่ $0 \leq i < N$, โปรเซส $(i + 1)$ จะเริ่มทำงานภายหลังโปรเซส i ทำงานเสร็จสิ้น

สำหรับโปรเซส 0 ถึง $N - 1$:

- `init` จะถูกเรียกหนึ่งครั้งพอดี.
- `strategy` จะถูกเรียก N^2 ครั้ง หลังจาก `init`.
 - รับประกันว่า สำหรับการเรียกครั้งที่ j , $b = j - 1$.

สำหรับโปรเซส N :

- `guess` จะถูกเรียกหนึ่งครั้งพอดี

โปรแกรมกรรมการ (judge) อาจจะสลับลำดับการเรียกกันระหว่างเกม แต่จะรับประกันว่าลำดับการเรียกโดยสัมพัทธ์ (relative) ของแต่ละเกมจะถูกรักษาไว้อย่างถูกต้อง

Constraints

- $1 \leq T \leq 400$.
- $2 \leq N \leq 30$.
- ผลรวมของ N^3 ในทุก ๆ เกมในแต่ละกรณีทดสอบจะไม่เกิน 27 000.
- $1 \leq K \leq N$
- $K \in \{1, 3, N\}$.
- $[P[0], P[1], \dots, P[N - 1]]$ จะเป็นการเรียงสับเปลี่ยนของ $[0, 1, \dots, N - 1]$.
- $0 \leq F[j] < N$ สำหรับแต่ละ j ที่ $0 \leq j < N^2$.
- i จะปรากฏเป็นจำนวน N ครั้งพอดีใน $[F[0], F[1], \dots, F[N^2 - 1]]$ สำหรับแต่ละ i ที่ $0 \leq i < N$.
- โปรแกรมกรรมการ (judge) จะไม่ได้ทำงานแบบปรับเปลี่ยนได้ (**not adaptive**) นั่นคือ ลำดับ $[P[0], P[1], \dots, P[N - 1]]$ และ $[F[0], F[1], \dots, F[N^2 - 1]]$ จะถูกกำหนดไว้ล่วงหน้า ก่อนการเรียกใช้ `init` ในครั้งแรก

Subtasks

Subtask	คะแนน	เงื่อนไขเพิ่มเติม
1	8	$K = 1, N = 2$.
2	7	$K = 1$ และ Bohan ตัดสินใจที่จะบอกแขกแต่ละคนว่าอยู่ที่ห้องใด.
3	14	$K = 1$ และ $[F[i \cdot N], F[i \cdot N + 1], \dots, F[i \cdot N + N - 1]]$ เป็นการเรียงสับเปลี่ยนของ $[0, 1, \dots, N - 1]$ สำหรับแต่ละ i ที่ $0 \leq i < N$.
4	18	$K = N$.
5	21	$K = 3, N \geq 3$.
6	32	$K = 1$.

Example

พิจารณาสถานการณ์ที่ $T = 1$ และ $N = 2, K = 2, P = [1, 0], F = [1, 0, 0, 1]$ เป็นเกมเดียวในกรณีทดสอบ

ให้ $S[i]$ แทนจำนวนชิ้นที่เหลือในถุง i เมื่อเริ่มต้น $S = [2, 2, 2, 2]$

สมมติว่าแขกผู้เข้าร่วมงานตัดสินใจว่าจะใช้กลยุทธ์ดังต่อไปนี้ก่อนที่จะเริ่มเกม

- สำหรับแขกในห้อง ถ้าเขาสามารถกินแพนเค้กต้นหอมในถุงได้
 - ถ้าแพนเค้กต้นหอมปัจจุบันเป็นแพนเค้กแรกที่เขากินได้ เขาจะกินจนหมด
 - ไม่เช่นนั้นจะกินแค่หนึ่งชิ้น
- สำหรับแขกนอกห้อง:
 - ถ้า $S = [0, 0, 1, 1]$, พวกเขาจะทายว่า $P = [1, 0]$
 - ไม่เช่นนั้น พวกเขาจะทายว่า that $P = [0, 1]$

เกมจะดำเนินไปดังนี้

ในขั้นแรก โปรแกรมกรรมการจะเริ่มโปรเซส 0 ที่แทนแขกในห้อง 0 แล้วเรียก $\text{init}(2, 2, 1, -1)$ หลังจากนั้น โปรแกรมกรรมการจะเรียกฟังก์ชันดังด้านล่าง:

Procedure call	Return value
$\text{strategy}(0, 1, 2)$	0
$\text{strategy}(1, 0, 2)$	2
$\text{strategy}(2, 0, 2)$	1
$\text{strategy}(3, 1, 2)$	0

สังเกตว่าการเรียกครั้งแรกและครั้งสุดท้ายจะต้องคืน 0 เพราะว่า $P[0] = F[0] = F[3] = 1$

หลังจากที่โปรเซสดังกล่าวทำงานจบ (สิ้นสุดรอบที่ 1) $S = [2, 0, 1, 2]$

ในขั้นถัดไป โปรแกรมกรรมการจะเริ่มโปรเซส 1 ที่แทนแขกในห้อง 1 แล้วเรียก $\text{init}(2, 2, 0, -1)$ หลังจากนั้น โปรแกรมกรรมการจะเรียกฟังก์ชันดังด้านล่าง:

Procedure call	Return value
$\text{strategy}(0, 1, 2)$	2
$\text{strategy}(1, 0, 0)$	0
$\text{strategy}(2, 0, 1)$	0
$\text{strategy}(3, 1, 2)$	1

สังเกตว่าการเรียกครั้งที่สองและสามจะต้องคืน 0 เพราะว่า $P[1] = F[1] = F[2] = 0$

หลังจากที่โปรเซสดังกล่าวทำงานจบ (สิ้นสุดรอบที่ 2) $S = [0, 0, 1, 1]$

สุดท้าย, โปรแกรมกรรมการจะเริ่มโปรเซส 2 ที่แทนแขกผู้ร่วมงานด้านนอก โปรแกรมกรรมการจะเรียกฟังก์ชันด้านล่าง:

Procedure call	Return value
<code>guess(2, 2, [1, 0, 0, 1], [0, 0, 1, 1])</code>	<code>[1, 0]</code>

แขกผู้ร่วมงานด้านนอกทนายการเรียงสับเปลี่ยนที่ถูกต้อง ดังนั้นกรณีทดสอบนี้จะถือว่าถูกต้อง

สังเกตว่า แนวทางด้านบนไม่ได้รับประกันว่าแขกด้านนอกจะทนายการเรียงสับเปลี่ยนได้ถูกต้องเสมอไป

นอกเหนือจากตัวอย่างนี้ ใน attachment package สำหรับโจทย์ข้อนี้ จะมีตัวอย่างข้อมูลนำเข้าและข้อมูลส่งออกเพิ่มเติมอีกหนึ่งตัวอย่าง

Sample Grader

เกรดเดอร์ตัวอย่าง รองรับเฉพาะการทดสอบแบบ **หนึ่งเกมต่อหนึ่งกรณีทดสอบ**

รูปแบบข้อมูลนำเข้า:

```
N K
P[0] P[1] ... P[N-1]
F[0] F[1] ... F[N*N-1]
reveal
```

- `reveal` มีค่าเป็น 0 หรือ 1.
 - ถ้า `reveal` เป็น 0, เกรดเดอร์จะทำงานเหมือนกรณีที่ Bohan จะไม่บอกหมายเลขห้องกับแขกคนใดเลย.
 - ถ้า `reveal` เป็น 1, เกรดเดอร์จะทำงานเหมือนกรณีที่ Bohan บอกหมายเลขห้องกับทุกคน.

ถ้าอาร์เรย์ที่คืนจาก `guess` ตรงกับ $[P[0], P[1], ..., P[N - 1]]$, เกรดเดอร์ตัวอย่างจะพิมพ์

```
Accepted
```

นอกจากนี้ เกรดเดอร์จะพิมพ์การเรียกฟังก์ชันต่าง ๆ เพื่อความสะดวกในการ debug